Laboratorio y prevención

Diapositivas de apoyo: Ejemplo práctico en Flask, prevención por capas y cierre. Ensayar el texto a 125-145 palabras por minuto. El tiempo de clics y cambios de ventana se suma al guion; preparar el laboratorio con ejemplo-flask.md antes de exponer.

Presentar el laboratorio

Llegamos a la demostración. Hasta ahora analizamos las causas y los riesgos; ahora veremos diferencias concretas entre una implementación vulnerable y una corregida. Utilizaremos Flask, una herramienta de Python para crear aplicaciones web pequeñas. La base de datos será SQLite y contendrá únicamente dos registros ficticios. Todo se ejecutará en la misma computadora, en la dirección de loopback. No estamos probando páginas de terceros ni usando cuentas reales. Ese alcance permite observar el mecanismo sin exponer información ni depender de una conexión externa.

Conectar las animaciones con la práctica real

Las comparaciones anteriores se reprodujeron dentro de las diapositivas. Vimos una búsqueda que devolvía datos ajenos, un comentario que producía un aviso falso, una solicitud que cambiaba un correo y una copia de sesión que permitía entrar como otra persona. Eran representaciones para reconocer cada causa y consecuencia.

Ahora cambiamos al laboratorio Flask. Aquí sí enviamos peticiones a un servidor local y comparamos rutas vulnerables con rutas corregidas. En XSS utilizaremos el aviso inocuo DEMO LOCAL: cambia el mensaje respecto de la animación, pero demuestra la misma capacidad de ejecutar una instrucción. No bloquearemos cuentas ni robaremos datos. La práctica observa la cookie, pero no reproduce el secuestro de una sesión.

Primera comparación: SQLi

Comenzamos con la búsqueda normal. Escribimos Ana y obtenemos un registro. Esto establece una referencia de funcionamiento legítimo. A continuación, usamos la entrada de prueba en la ruta vulnerable. El resultado incluye los dos registros ficticios porque la cadena cambia la condición de la consulta construida por concatenación. No es necesario leer cada símbolo en voz alta. Señalemos que la entrada dejó de actuar solamente como un nombre y participó en la instrucción que recibe la base de datos. Esa es la causa que queremos identificar.

Ahora enviamos exactamente el mismo valor a la ruta corregida. La aplicación busca la cadena completa como nombre y devuelve una lista vacía. Después volvemos a buscar Ana para comprobar que la función sigue sirviendo para su objetivo original. En el editor, comparamos la consulta que pega texto con la que utiliza un parámetro. El signo de interrogación deja reservado el lugar del valor y ese valor se entrega aparte. La diferencia importante no es que bloqueamos una frase específica, sino que mantuvimos separadas las instrucciones y los datos.

Segunda comparación: XSS

Pasamos a la página de saludo. En la versión vulnerable, la entrada se une directamente al HTML de la respuesta. Utilizamos un script de prueba que únicamente muestra una alerta con el texto DEMO LOCAL. Si aparece, vemos que el navegador ejecutó contenido aportado como entrada. Cerramos la alerta y repetimos el valor en la ruta segura. En esa respuesta, una plantilla fija de Jinja recibe la entrada como variable y la muestra escapada dentro del contexto HTML. La cadena aparece como texto y no se ejecuta la alerta.

La demostración no recopila cookies, no envía información a otro sitio y no necesita hacerlo. Una ejecución visible basta para explicar la frontera que se rompió. Es importante no concluir que todas las formas de XSS se corrigen con una única transformación de texto. El contexto importa. Aquí mostramos un valor dentro de texto HTML; otros contextos requieren otros controles. Volvemos a la idea principal: si queremos un saludo, debemos presentar un saludo, no otorgar a la entrada la capacidad de cambiar la estructura o el comportamiento de la página.

Tercera comprobación: CSRF y cookie

Abrimos el perfil ficticio y cambiamos el alias mediante el formulario. La operación válida se acepta. Después, en las herramientas del navegador, quitamos el campo oculto del token CSRF y enviamos de nuevo. Flask-WTF rechaza la petición con un estado 400 porque falta la comprobación esperada. Recargar restaura el formulario válido. Expliquemos con precisión lo que vimos: comprobamos que el servidor rechaza una solicitud sin token. No hemos reproducido un ataque completo desde otra página y no debemos describirlo como si lo hubiéramos hecho.

En la sección de almacenamiento del navegador podemos observar la cookie de sesión del laboratorio. Revisamos HttpOnly y SameSite. Secure está desactivado solamente porque esta prueba utiliza HTTP en la propia computadora; en producción con HTTPS debe activarse. No mostraremos contenido de sesiones reales ni claves. También recordamos que la sesión estándar de Flask está firmada, no cifrada. La inspección sirve para conectar los términos de la exposición con opciones concretas, no para afirmar que esta aplicación de demostración incorpora todos los controles de una plataforma real.

Convertir el ejemplo en una lista de prevención

Volvemos a las diapositivas. La primera capa es la entrada: comprobar tipo, longitud y formato. Si esperamos una edad, no aceptar cualquier texto; si esperamos una elección, limitarla a opciones definidas. Esto mejora la calidad de los datos, pero no sustituye los controles posteriores. La segunda capa es la base de datos: consultas parametrizadas y permisos adecuados. También hay que comprobar que la persona pueda acceder al recurso concreto. Saber quién inició sesión no autoriza por sí solo a consultar cualquier registro.

La tercera capa es la salida. Mostramos información de forma segura para el contexto donde aparece. Evitamos convertir entradas en HTML ejecutable sin necesidad y utilizamos las protecciones de las plantillas correctamente. La cuarta capa son las acciones: comprobamos tokens CSRF y autorización en el servidor cuando corresponde. Ninguna comprobación importante debería depender solamente de ocultar un botón. La interfaz ayuda al usuario, pero el servidor debe tomar la decisión aunque alguien construya una solicitud por su cuenta.

La quinta capa es la sesión: HTTPS, atributos de cookies, duración y mecanismos apropiados de invalidación. La sexta es la operación diaria: mantener dependencias, revisar cambios y registrar eventos sin secretos. Un sistema no queda protegido para siempre porque una demostración haya funcionado. Se modifica el código, aparecen nuevas funciones y cambian los flujos. La seguridad debe acompañar ese mantenimiento. Tampoco necesitamos prometer que una lista elimina todos los riesgos; su valor es convertir principios generales en decisiones que un equipo puede revisar y comprobar.

Cierre del grupo

Podemos cerrar con una comparación que reúne las partes del grupo. SQLi enseña a separar los valores de las órdenes de base de datos. XSS enseña a separar el contenido de las instrucciones del navegador. CSRF enseña a no confundir una sesión presente con una acción intencional. La protección de sesiones enseña que una credencial temporal también necesita transporte seguro, límites y un final. Los controles se relacionan, pero cada uno responde a una pregunta diferente. Comprender esa diferencia nos ayuda a elegir una solución con fundamento.

La seguridad web se diseña en decisiones pequeñas que se repiten: qué aceptamos, dónde lo usamos, qué mostramos y qué permitimos cambiar. No depende de que una página parezca sofisticada ni de añadir símbolos de candado. Depende de conservar el significado de los datos y verificar la confianza en cada paso. Con esto termina nuestra exposición. Muchas gracias.

Apoyo para ensayar y contingencia

Ensayar cambios entre presentación, editor y navegador con Alt+Tab. Preparar las URL desde ejemplo-flask.md y ampliar el editor para que se lea. Si el entorno falla, usar capturas tomadas durante un ensayo y decir que son del laboratorio; no presentarlas como una ejecución en vivo. El mínimo de duración se calcula con el monólogo del grupo. Las transiciones y la demostración aportan margen adicional, no reemplazan contenido. El cierre termina con Gracias, sin ronda de preguntas ni anuncio de recursos.

Indicaciones para presentar esta parte

La diapositiva Ejemplo práctico en Flask conserva solo el título. Cambiar con Alt+Tab al laboratorio ya iniciado y seguir ejemplo-flask.md. Las animaciones no se controlan con el mouse; el laboratorio local real sí requiere usar sus formularios. Al volver, explicar la prevención por capas y terminar en Gracias. No mostrar el menú de recursos frente al público.

Diccionario de términos

Flask: Herramienta de Python para construir aplicaciones web.

SQLite: Motor de base de datos integrado en Python; el ejemplo utiliza datos en memoria.

Entorno virtual: Carpeta que separa las bibliotecas de este proyecto de otras instalaciones.

Dependencia: Biblioteca externa necesaria para ejecutar un programa.

Loopback: Dirección que apunta a la propia computadora, como 127.0.0.1.

Puerto: Número que identifica el servicio local; el lanzador elige uno libre.

DevTools: Herramientas del navegador para inspeccionar HTML, solicitudes y cookies.

Datos ficticios: Registros inventados para demostrar sin utilizar información real.

Defensa por capas: Combinación de controles que protegen diferentes partes del recorrido de la información.